综合论文训练

AIoT 操作系统性能优化关键技术研究

系　　别：计算机科学与技术系

专　　业：计算机科学与技术

姓　　名：王　　铮

指导教师：陈　　渝　副教授

二〇二六年三月

关于论文使用授权的说明

本人完全了解清华大学有关保留、使用综合论文训练论文的规定，即：学校有权保留论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

作者签名：　　导师签名：

日　　期：　　日　　期：

摘　要

【待完善】填写中文摘要

关键词：AIoT；操作系统；性能优化；ArceOS；Starry

Abstract

【待完善】 填写英文摘要

Keywords: AIoT; Operating System; Performance Optimization; ArceOS; Starry

目 录

第 1 章 绪论 1
1.1 研究背景 1
1.1.1 操作系统内核架构的演进 1
1.1.2 组件化操作系统与 ArceOS 1
1.1.3 Starry OS 的发展 2
1.2 相关工作 2
1.3 本文工作 2
1.4 论文结构安排 2
第 2 章 系统背景与总体架构 4
2.1 设计目标与需求分析 4
2.1.1 AIoT 场景需求 4
2.1.2 StarryOS 的设计目标 4
2.2 ArceOS 基础与 StarryOS 定位 5
2.2.1 ArceOS 的灵活内核理念 5
2.2.2 ArceOS 的组件层次结构 5
2.2.3 StarryOS 的系统定位 6
2.3 系统总体架构 6
2.3.1 总体分层与模块关系 6
2.3.2 组件边界与仓库组织 7
2.4 内核执行模型 7
2.4.1 用户态与内核态的基本切换流程 7
2.4.2 进程、线程与任务对象关系 7
2.5 核心资源管理对象 7
2.5.1 地址空间与内存对象 7
2.5.2 文件、设备与 I/O 对象 8
2.5.3 通信与事件对象 8
2.6 本章小结 8
第 3 章 核心子系统设计与实现 9
3.1 系统调用框架 9
3.2 任务与进程管理 9

3.3 内存管理 10
3.4 文件系统 10
3.5 网络与终端子系统 10
3.6 信号与同步机制 11
3.7 本章小结 11
第 4 章 关键机制与性能优化 12
4.1 Rust 组件化开发的挑战 12
4.2 extern_trait 机制 12
4.2.1 设计动机 12
4.2.2 实现原理 13
4.2.3 应用场景 13
4.3 用户态切换与反转控制流 13
4.3.1 传统宏内核控制流的局限 13
4.3.2 反转控制流设计 13
4.3.3 基于 axcpu 的实现 13
4.4 高效内核-用户交互机制 14
4.4.1 用户内存访问机制 14
4.4.2 缓冲区抽象设计 14
4.5 异步 I/O 机制 14
4.5.1 设计目标 14
4.5.2 同步与异步混合架构 15
4.5.3 与信号和硬件的集成 15
4.6 多级缓存优化 15
4.7 多核调度与虚拟内存优化方向 15
4.7.1 多核调度优化 15
4.7.2 虚拟内存优化 15
4.8 本章小结 15
第 5 章 AI 应用支持探索 16
5.1 目标平台与硬件环境 16
5.2 NPU 驱动设计 16
5.3 AI 应用运行支持 16
5.4 当前进展与后续计划 16
第 6 章 实验评测 17
6.1 实验环境 17

6.2 功能完整性评测 17
6.2.1 系统调用测例 17
6.2.2 应用兼容性测试 17
6.3 性能评测 17
6.3.1 系统调用开销 18
6.3.2 I/O 性能 18
6.4 优化效果分析 18
6.5 开发工作量统计 18
6.6 本章小结 18
第 7 章 总结与展望 19
7.1 工作总结 19
7.2 未来工作展望 19
参考文献 20
附录 A 外文资料的调研阅读报告（或书面翻译） 21
附录 B 其他附录内容 22
致 谢 23
声 明 24

插图清单

附表清单

符号和缩略语说明

AIoT	Artificial Intelligence of Things，人工智能物联网
OS	Operating System，操作系统

第 1 章 绪论

1.1 研究背景

1.1.1 操作系统内核架构的演进

宏内核、微内核和Unikernel是操作系统研究中最具代表性的三类内核架构。宏内核将进程管理、内存管理、文件系统、网络协议栈和设备驱动等大多数核心功能置于同一特权地址空间内实现，其优势在于模块间调用路径短、运行时开销低，因而长期在通用计算场景中占据主导地位；但其缺点也较为明显，即系统规模扩大后内部耦合增强，故障隔离与安全边界相对薄弱。微内核则将尽可能少的机制保留在内核态，仅在内核中维护地址空间、线程调度和进程间通信等最基本功能，而将文件系统、驱动等服务下沉到用户态。该设计有利于提高可靠性、可维护性和隔离性，但频繁的跨边界通信会带来额外开销，实现复杂度也更高。

Unikernel 则进一步面向特定应用进行裁剪，将应用与所需的最小操作系统服务整体编译为单一镜像运行。与通用操作系统相比，Unikernel能够显著减少系统体积、缩短启动时间，并降低运行路径中的冗余功能开销，因而在云原生、边缘设备和专用化部署场景中具有吸引力。然而，这种架构通常以牺牲通用性和兼容性为代价，应用迁移与调试支持也相对不足。总体来看，三类架构分别在性能、隔离性、灵活性和兼容性之间做出不同权衡，并不存在适用于全部场景的统一最优方案。随着 AIoT、云边协同等场景对系统形态提出多样化需求，如何在不同架构之间复用基础能力、降低定制成本，成为操作系统设计中的一个重要问题。

1.1.2 组件化操作系统与 ArceOS

组件化操作系统的核心思想，是将内核中的基础服务按照清晰的接口边界拆分为可组合、可替换、可复用的独立组件，从而降低系统耦合度并提升定制效率。与传统以单一内核形态为中心进行扩展的方式相比，组件化设计更强调将调度、内存、文件系统、设备抽象等共性能力沉淀为稳定模块，再根据具体场景组织成不同系统形态。对于需要在性能、隔离性、兼容性和部署成本之间反复权衡的现代应用场景而言，这种设计方法具有较高的工程价值。

ArceOS 正是在这一背景下提出的实验内核。其基本出发点并非围绕某一种固定架构优化，而是将不同内核形态共享的底层服务视为“内核意义上的库”，通过统一的组件边界和初始化后执行逻辑的差异，构造出 Unikernel、宏内核乃

至 Hypervisor 等不同系统形态。这一设计理念可概括为“One Architecture to build all”，即通过一套统一的组件化基础设施支撑多种内核形态的快速构建。该思路为后续在 ArceOS 基础上扩展兼容性内核提供了方法论基础，也为在不同场景间复用已有内核能力提供了新的实现路径。

1.1.3 Starry OS 的发展

【待完善】梳理 Starry-Next → Starry Mix → StarryOS 主线的演进历程

1.2 相关工作

【待完善】每类 2-3 句话即可：

- 其他 Rust OS：Asterinas（Framekernel）、Theseus、Tock
- Linux 兼容性内核：gVisor、Graphene/Occlum（TEE 场景）
- 性能优化相关：io_uring、CortenMM（SOSP’25 最佳论文）

1.3 本文工作

【待完善】分 3-4 段简述本文主要工作与贡献，建议分成两层表述：

1. 完整兼容性内核实现：在 ArceOS 基础上实现 StarryOS，支持进程、内存、文件系统、网络、终端、信号等核心能力，并实现 200+ Linux 系统调用
2. 关键机制与性能优化：围绕组件化兼容内核中的关键问题，设计并实现 extern_trait、用户态切换与反转控制流、高效内核-用户交互、异步 I/O、多级缓存等机制
3. AI 应用支持探索：面向 AIoT 场景，探索 NPU 驱动与 AI 应用运行支持方向

1.4 论文结构安排

全文共分为七章。第 1 章介绍研究背景、相关工作以及本文的主要研究内容。第 2 章从 AIoT 场景需求与 ArceOS 基础出发，介绍 StarryOS 的系统定位、设计目标与总体架构。第 3 章进一步展开兼容性内核核心子系统的设计与实现，包括系统调用、任务与进程管理、内存管理、文件系统、网络与终端以及信号与同步机制。第 4 章围绕组件化兼容内核实现中的关键技术问题，介绍 extern_trait、用户态切换与反转控制流、高效内核-用户交互、异步 I/O 以及多级缓存等机制与优化。第 5 章

讨论面向 AIoT 场景的 AI 应用支持探索。第 6 章从功能完整性、应用兼容性和性能等维度对系统进行实验评测。第 7 章对全文工作进行总结，并展望后续研究方向。

第 2 章 系统背景与总体架构

本章首先从 AIoT 场景对操作系统的共性需求出发，说明 StarryOS 的问题背景与设计目标；随后结合 ArceOS 的组件化基础与灵活内核理念，给出 StarryOS 的系统定位，并从总体分层、执行模型和核心资源对象三个层面概述系统结构。通过本章的介绍，可以将后续各核心子系统实现以及关键机制优化放回统一的系统框架中进行理解。

2.1 设计目标与需求分析

2.1.1 AIoT 场景需求

AIoT 场景兼具通用计算与嵌入式计算的双重特征。一方面，边缘侧设备往往需要运行日益复杂的应用框架、网络协议和数据处理逻辑，这要求操作系统具备较好的接口兼容性与应用承载能力；另一方面，设备端的算力、内存和能耗预算通常受到严格限制，系统又必须尽可能降低运行时开销，并对 I/O、调度和内存访问路径进行精细优化。因此，面向 AIoT 场景的操作系统不能仅追求功能完整，也不能仅满足轻量化部署，而需要在兼容性、性能和资源利用率之间取得平衡。

与此同时，AIoT 部署环境通常呈现出硬件平台多样、应用需求变化快以及软硬件协同紧密等特点。操作系统不仅需要支持跨架构移植和设备适配，还应具备较强的模块化与可扩展能力，以便随着应用场景变化快速裁剪、替换或新增内核功能。尤其当系统需要进一步支持 NPU、DMA、高速通信接口等专用硬件能力时，内核内部的接口边界是否清晰、关键子系统是否便于复用，都会直接影响后续演进成本。基于这些考虑，AIoT 场景中的操作系统设计天然需要兼顾组件化、兼容性与性能优化三方面目标。

2.1.2 StarryOS 的设计目标

StarryOS 的首要目标，是在 ArceOS 组件化基座之上构建一个具备较强 Linux 应用兼容能力的宏内核系统，使其能够承载更丰富的用户态程序、开发工具链和典型服务程序运行。与单纯面向专用化任务的内核不同，兼容性目标要求系统在进程管理、文件系统、信号、网络、终端和事件机制等方面逐步对齐 Linux/POSIX 语义，从而使现有软件生态能够以较低成本迁移到系统之上。

在兼容性之外，StarryOS 还强调组件化复用与性能优化并重。一方面，系统需要延续 ArceOS 的组件化思想，将与具体内核形态弱相关或可抽象的能力拆分为独

立模块，以降低工程耦合并支持后续开源复用；另一方面，兼容性内核在系统调用路径、用户态访存、I/O 处理和缓存管理等位置存在明显性能压力，因此需要围绕关键路径进行专门优化。进一步地，面向 AIoT 场景的后续演化还要求系统具备良好的跨架构移植能力以及接入专用硬件与 AI 应用运行时的潜力。因此，StarryOS 的总体目标可以概括为：在组件化基础上实现完整兼容性，并在此基础上持续追求工程可维护性与系统性能。

2.2 ArceOS 基础与 StarryOS 定位

2.2.1 ArceOS 的灵活内核理念

ArceOS 的设计出发点，是避免将操作系统定制工作绑定在某一种固定内核架构之上。传统定制内核通常围绕单一形态展开，当应用需求从专用化运行转向更强的隔离性或更丰富的兼容性时，已有工作往往难以直接迁移，只能重新开发或退回通用内核。为降低这一迁移成本，ArceOS 提出了灵活内核的设计原则，即将不同内核形态共享的基础服务抽象为可复用组件，而将内核最终表现出的形态视为初始化完成后执行逻辑的结果。这样一来，同一套底层能力便可以被组织为 Unikernel、宏内核或 Hypervisor 等不同系统。

在实现路径上，ArceOS 采用组件化方法构建硬件抽象、运行时支撑和上层功能模块，并强调组件之间通过明确接口进行协作。该设计的价值不在于预先规定唯一正确的系统结构，而在于为不同内核形态保留足够的组合空间，使得上层系统能够在共享底座能力的前提下针对目标场景进行定制。对于本文所研究的 StarryOS 而言，ArceOS 提供的不仅是一个可复用的底层基座，更是一种将兼容性内核实现建立在组件化原则之上的方法论基础。

2.2.2 ArceOS 的组件层次结构

从实现组织上看，ArceOS 及其上层扩展通常可以分为若干层次。最底层是与硬件平台和执行环境直接相关的基础支撑，负责硬件抽象、系统初始化以及运行时支持；在其之上是围绕内存管理、任务调度、文件系统、网络、设备驱动等组织的核心功能模块；再向上则是一些能够在不同内核形态中复用的通用组件；最上层才是具体系统形态对这些能力的组合与使用。这样的层次划分有助于将“底层公共能力”和“上层系统策略”分离开来。

对于本文所讨论的系统而言，也可以沿着这一思路来理解其构建路径：底层复用组件化基座提供的硬件抽象与运行时能力，中间层承载内存、任务、文件系统、

网络等基本操作系统功能，而上层则在此基础上实现面向 Linux 兼容性的宏内核功能。后文可据此给出一张总体层次结构图，以说明各层之间的依赖关系。

2.2.3 StarryOS 的系统定位

StarryOS 可以看作 ArceOS 在 Linux 兼容性宏内核方向上的一条重要扩展路径。与 ArceOS 原本更强调灵活内核与组件复用的实验性基座定位不同，StarryOS 更直接地面向用户态程序运行环境，要求系统能够提供完整的进程抽象、地址空间管理、文件与设备抽象、网络接口以及信号与事件处理机制，从而承载更接近 Linux 使用习惯的软件生态。因此，StarryOS 并不是对 ArceOS 基础模块的简单堆叠，而是在复用底座能力的同时，将大量 POSIX/Linux 兼容语义逐步落实到具体实现中。

对于本文而言，更重要的并不是工程目录如何命名，而是系统在结构上如何划分职责边界。整体上看，StarryOS 在下层继承了组件化基座提供的通用能力，在上层围绕兼容性需求组织进程、内存、文件、网络和信号等核心功能，并通过若干相对独立的模块承载特定语义。这种定位既保留了组件化设计带来的可复用性，也使系统能够朝着完整兼容性内核的方向持续演化。

2.3 系统总体架构

2.3.1 总体分层与模块关系

从总体上看，StarryOS 的结构可以概括为“组件化基座 + 宏内核主体 + 若干相对独立的功能模块”三部分。底层部分负责提供硬件抽象、运行时支撑以及调度、内存、文件系统、网络等基础能力；中间的宏内核主体负责组织用户程序加载、系统调用分发以及多个子系统之间的协作；上层则围绕兼容性需求实现进程管理、信号处理、虚拟内存访问等具体功能。这样的组织方式使系统既能复用底层能力，又能在主体部分集中实现 Linux 兼容性相关逻辑。

从功能关系上看，系统调用层位于用户态与内核态交互路径的中心位置，负责将陷入内核的请求分派到任务管理、内存管理、文件系统、网络与信号等子系统；文件系统和网络子系统又分别通过统一的对象抽象与设备、缓存和事件机制发生关联；任务管理与调度机制则为整个系统提供执行上下文；信号、poll/epoll、futex 等同步与事件相关机制横向贯穿多个子系统，构成阻塞、唤醒和中断语义的统一基础。后续章节将沿着这些模块边界，分别展开对核心子系统和关键机制的介绍。

2.3.2 组件边界与仓库组织

StarryOS 的工程组织并未将全部功能集中在单一仓库中，而是延续了 ArceOS 体系中较为明显的组件化拆分思路。根据较新的设计文档，完整系统包含 30 余个相关代码仓库，其中一部分面向底层基础能力，一部分面向宏内核特有功能，还有一部分则是为了提升通用性而单独抽离的工具型 crate。这样的拆分并不意味着每个仓库都完全独立演化，而是强调通过接口隔离来控制依赖方向，使不同层次的功能能够在尽可能小的影响范围内迭代。

从当前代码形态看，部分旧文档中分散在多个 crate 中的职责已经向 `starry-kernel` 收敛，但仓库级组件化的基本原则并未改变：与具体系统形态弱相关、适合复用的能力尽量独立出来，而与当前宏内核实现强绑定的逻辑则在内核主体中集中维护。对论文写作而言，这一点意味着在描述系统结构时，重点应放在“功能边界如何划分”和“依赖关系如何控制”上，而不必过度拘泥于某一时刻仓库数量或目录布局的微小变化。

2.4 内核执行模型

2.4.1 用户态与内核态的基本切换流程

【待完善】 说明系统调用、异常与中断在 StarryOS 中的基本处理流程， 给出从用户态陷入内核、处理完成后返回用户态的整体路径。

2.4.2 进程、线程与任务对象关系

【待完善】说明进程、线程、任务在系统中的映射关系， 以及这些对象在资源管理中的职责划分。

2.5 核心资源管理对象

【待完善】 作为开头说明：本节从资源对象角度概括 StarryOS 的组织方式， 为后续各子系统实现章节做铺垫。

2.5.1 地址空间与内存对象

【待完善】 概述地址空间、页表、映射区域等核心对象及其关系。

2.5.2 文件、设备与 I/O 对象

【待完善】概述文件描述符、file、inode、dentry、设备对象在系统中的组织方式。

2.5.3 通信与事件对象

【待完善】概述 socket、poller、waker、signal 等对象在事件驱动与进程交互中的作用。

2.6 本章小结

本章从 AIoT 场景需求和 ArceOS 的灵活内核理念出发，说明了 StarryOS 所面对的问题背景以及系统设计所需满足的若干目标。随后，本文将继续从核心子系统的角度展开，具体介绍兼容性内核在系统调用、任务管理、内存管理、文件系统以及网络等方面的组织方式与实现基础。

第 3 章 核心子系统设计与实现

本章在第 2 章总体架构的基础上，进一步介绍 StarryOS 兼容性内核中若干核心子系统的组织方式与实现基础。与第 4 章聚焦关键机制和性能优化不同，本章更关注系统作为一个完整 Linux 兼容性内核所必须具备的“基本盘”，即系统如何承载用户程序的执行、如何管理地址空间与文件对象、如何处理网络与终端输入输出，以及如何通过信号与同步机制组织进程间交互。通过这些内容，可以更清晰地理解后续各项优化工作是建立在怎样的系统基础之上的。

3.1 系统调用框架

系统调用框架是 StarryOS 连接用户态程序与内核子系统的核心入口。当用户程序通过陷入指令进入内核后，系统首先根据处理器提供的上下文信息识别系统调用号及参数，随后进入统一的系统调用分发路径，再由内核将请求转交给文件、内存、任务、时间、信号等具体子系统处理，最后将返回值写回用户上下文并恢复执行。从设计目标上看，该框架既要尽可能贴近 Linux 的系统调用语义，又要避免将过多具体逻辑堆叠在单一路径中，从而保持模块边界清晰。

从当前代码组织来看，系统调用相关逻辑主要集中在 `starry-kernel` 的 `syscall` 模块下，并按照资源、信号、时间等主题进一步划分子模块。这种组织方式体现出一种较为稳妥的折中：对外保持统一的系统调用入口，对内则按照功能域将实现分散到相对独立的模块中，减少不同子系统之间的直接耦合。对于兼容性内核而言，这种结构也更有利于逐步补齐系统调用支持范围，并在后续演化中持续维护语义一致性。从实现组织来看，系统调用相关逻辑采用了“统一入口、按功能域分解实现”的方式：对外保持一致的系统调用进入路径，对内则按照资源、信号、时间等主题划分子模块，减少不同子系统之间的直接耦合。对于兼容性内核而言，这种结构也更有利于逐步补齐系统调用支持范围，并在后续演化中持续维护语义一致性。

3.2 任务与进程管理

【待完善】简述 fork/exec/wait/exit/clone 等核心进程管理系统调用的实现要点，重点说明与 ArceOS 任务模型的对接方式，以及任务、线程、进程对象的组织。

3.3 内存管理

【待完善】 简述 mmap/munmap/brk 等的实现，涵盖匿名映射、文件映射、懒分配、CoW 等机制。

3.4 文件系统

StarryOS 的文件系统设计遵循类 Unix 系统常见的 VFS 分层思路，即通过统一抽象将具体物理文件系统、伪文件系统以及设备文件纳入同一访问框架。在这一框架下，文件描述符负责向进程暴露统一的访问句柄，`file` 对象承载一次打开后的状态，`inode` 与目录项等对象则承担对底层命名空间和存储实体的组织。这种设计的意义在于，将进程看到的文件操作接口与具体文件系统实现解耦，从而能够在相同系统调用语义下支持普通磁盘文件、管道、套接字以及 `/proc`、`/dev` 等特殊文件。

结合较新的设计文档与当前代码结构可以看到，StarryOS 侧既需要处理普通文件系统，也需要实现 `procfs`、`tmpfs`、`devfs` 等伪文件系统支持，因此 VFS 层不仅是功能复用的基础，也是后续缓存优化的重要承载点。页缓存、块缓存和目录项缓存等机制虽然将在后文重点讨论，但它们能够发挥作用的前提，正是系统已经建立起统一的文件对象与路径解析框架，使不同来源的数据访问都可以被纳入一致的管理路径中。

3.5 网络与终端子系统

网络与终端子系统共同构成了 StarryOS 面向真实用户空间应用的重要外部交互能力。根据设计文档，系统网络栈建立在 `smoltcp` 之上，并在其外封装 POSIX 风格的 socket 抽象，以支持 TCP、UDP 和 Unix Domain Socket 等常见通信方式；同时，系统还逐步补齐了多设备接入和与事件机制协同工作的能力。当前代码中，网络相关逻辑主要分布在文件抽象和底层网络模块的结合处，这也反映出“套接字即文件对象”的 Unix 风格设计思路。

终端相关能力则主要面向交互式应用和开发工具链支持。为了运行 shell、编辑器以及更复杂的图形与文本应用，系统需要提供 tty/pty、行规程以及作业控制等基础机制，使应用能够在接近 Linux 的终端语义下运行。这些能力本身未必体现出特别强的创新性，但它们对于兼容性内核的完整性至关重要，也是后续展示 Alpine Linux、开发工具链和图形环境支持时不可缺少的系统基础。

3.6 信号与同步机制

信号与同步机制贯穿于 StarryOS 的多个子系统之中，是实现类 Linux 进程交互语义的重要支撑。信号机制负责在异步事件发生时向目标进程或线程传递通知，并要求系统在合适的边界上检查待处理信号、构造用户态处理上下文并恢复执行；同步机制则需要为进程阻塞、等待和唤醒提供统一基础，包括 futex、poll/select/epoll 等常见设施。两者虽然分别服务于不同语义目标，但在兼容性内核内部往往共享一套事件等待与唤醒路径。

从系统组织角度看，信号机制并不是一个孤立模块，而是与系统调用返回路径、任务状态切换以及 I/O 事件处理紧密结合。特别是在阻塞系统调用场景下，信号能否正确打断等待过程、系统是否能够按照预期返回错误码或执行重启语义，直接决定了用户态应用的兼容性表现。也正因如此，本文后续在讨论异步机制时，还需要进一步说明信号与事件唤醒路径之间的配合关系。

3.7 本章小结

【待完善】总结本章对兼容性内核基本盘的实现，说明这些基础子系统为后续关键机制与性能优化提供了承载平台。

第 4 章 关键机制与性能优化

在完成兼容性内核基本功能之后，StarryOS 面临的主要问题不再只是“是否具备某项能力”，而是“这些能力能否以可维护、可复用且足够高效的方式组织起来”。一方面，ArceOS/StarryOS 所坚持的组件化路线要求系统在 Rust 严格的依赖与类型约束下处理复杂的跨模块协作；另一方面，Linux 兼容性内核在用户态访存、I/O 路径、事件等待与缓存管理上天然存在显著的性能压力。因此，本章聚焦于若干真正影响系统结构和性能上限的关键机制，说明系统在组件化实现与性能优化之间所做的设计取舍。

4.1 Rust 组件化开发的挑战

Rust 的模块与包管理机制天然强调依赖关系在编译期的静态确定，这使其在构建大型系统时能够获得良好的类型安全与优化效果，但也会给组件化操作系统开发带来特殊挑战。对于内核而言，某些功能依赖并不总是沿着“底层依赖上层”的单向路径展开。例如，底层组件有时需要回调上层提供的能力，或者在抽象数据结构中挂接由上层定义的扩展状态；再如 `#[panic_handler]`、`#[global_allocator]` 这类入口级能力，往往天然表现出一种“反向依赖”的特征。与 C 语言更多在链接阶段解决符号关系不同，Rust 需要在更早阶段就将这些依赖明确下来，因此会更早暴露出组件边界不清或依赖方向不合理的问题。

这一问题在操作系统开发中尤为突出。操作系统内部模块数量多、层次深，而且控制流和资源关系往往比普通应用程序更复杂，简单地通过重新组织调用顺序并不能完全消除循环依赖或跨层扩展需求。因此，如何在不破坏 Rust 类型系统和性能优势的前提下，为上层模块向下层注入实现、为下层模块保留可扩展接口，就成为组件化宏内核设计中的一个核心问题。后文介绍的 `extern_trait` 机制，正是在这一背景下提出的。

4.2 extern_trait 机制

4.2.1 设计动机

【待完善】简述 crate_interface 解决的问题（跨 crate 的接口定义与实现解耦），指出其局限性（不支持复杂类型传递、RAII 语义受限）， 对比 `Box<dyn Trait>`、`*mut ()` 等方案的不足， 引出 extern_trait 的设计目标。

4.2.2 实现原理

【待完善】 说明 extern_trait 的核心思想：通过过程宏在编译期生成调用桩， 实现零开销的跨 crate 接口调用，支持任意类型和 RAII 语义。可附代码示例。

4.2.3 应用场景

【待完善】列举在 StarryOS 中的具体应用（TaskExt、on_enter/on_leave/drop 等），以及已发布/计划发布到 crates.io 的情况（extern-trait crate）。

4.3 用户态切换与反转控制流

4.3.1 传统宏内核控制流的局限

传统宏内核通常将系统执行过程组织为“用户程序调用内核服务”的形式：用户态程序通过系统调用主动进入内核，内核在完成请求后再返回用户态继续执行。这种模式在概念上直观、实现上成熟，也是大多数经典操作系统教材默认采用的控制流描述方式。然而，当系统希望将更多控制流相关逻辑抽象为可复用组件时，这种结构会暴露出局限性。原因在于，内核虽然能够在陷入时临时接管控制权，却很难以统一方式主动组织用户程序执行的不同阶段，因此很多与进入用户态、离开用户态、任务切换相关的逻辑容易分散在不同位置，增加模块间耦合。

对于强调组件化的 StarryOS 而言，这种局限尤其明显。如果内核只能被动响应用户程序的调用，那么一些原本应当由统一机制处理的控制流操作，往往需要通过回调、全局状态或额外约定散落在不同模块中，不利于形成清晰边界。也正因为如此，StarryOS 在用户态切换机制上采取了更强调内核主动组织控制流的设计思路，从而为任务扩展、资源切换和后续的上下文管理提供更整洁的承载方式。

4.3.2 反转控制流设计

【待完善】说明 StarryOS 的设计：内核代码“调用”用户程序，主动控制用户程序流， 对比 Asterinas 的类似思路，说明其对组件化架构的意义。

4.3.3 基于 axcpu 的实现

【待完善】简述 axcpu 组件的职责（封装用户态/内核态切换的底层机制）， 说明实现难点及关键设计决策。

4.4 高效内核-用户交互机制

4.4.1 用户内存访问机制

【待完善】说明内核访问用户内存的设计目标——零额外开销（无异常时）。介绍 Exception Fixup 机制：在汇编层面处理页异常并保证控制流不受破坏，参考 Linux 的实现思路，在没有异常发生时不存在额外开销。

4.4.2 缓冲区抽象设计

【待完善】说明传统 `read/write` 接口（`&mut [u8]`）在处理用户数据时需要额外分配与拷贝的问题。介绍 Buf/BufMut 抽象：统一内核与用户缓冲区，大部分场景实现 one-copy I/O，说明其与 Linux 设计的契合性，以及更 Rusty 的接口特点。

4.5 异步 I/O 机制

4.5.1 设计目标

在 StarryOS 中，引入异步机制的目的并不是为了追求某种编程范式上的“新颖性”，而是为了解决同步实现中广泛存在的低效轮询问题。对于串口、网络设备、输入设备以及 `futex`、`waitpid` 等需要等待事件发生的操作，如果仍然采用频繁检查状态的同步路径，系统会在 I/O 瓶颈场景下浪费大量处理器时间，并显著拉低整体吞吐量。因此，异步机制的直接目标，是在真正存在等待需求的路径上尽量减少无效轮询，使阻塞中的任务在事件发生时被精确唤醒。

不过，异步并不天然等于高性能。纯同步实现的优点在于控制流直接、上下文切换成本较低，但复杂等待场景往往难以优雅处理；纯异步实现则更容易统一事件机制，也便于复用 Rust 异步生态中的部分思路，但往往伴随更高的状态机复杂度、额外堆内存使用以及潜在性能损失。基于这一判断，StarryOS 更倾向于在整体上保持同步任务调度框架，而在真正耗时且需要等待事件的路径上引入异步兼容层，形成同步与异步混合的设计。这也成为后续 Poller、Pollable 与 Waker 机制设计的出发点。

4.5.2 同步与异步混合架构

【待完善】介绍 Waker/Poller 机制：PollSet（无锁单向链表存储 Waker）、Pollable 接口（poll + register），以及 Poller 实现的核心逻辑。 说明如何兼容 POSIX poll 语义。可附架构图。

4.5.3 与信号和硬件的集成

【待完善】 说明异步机制与信号（可中断的系统调用）的交互， 以及外设中断（VirtIO 网卡/输入、UART）通过注册 Waker 实现硬件唤醒的机制。

4.6 多级缓存优化

【待完善】 展开说明页缓存（page cache）、块缓存（block cache）、目录项缓存（dcache）三层缓存的实现细节，以及脏页写回策略。

4.7 多核调度与虚拟内存优化方向

4.7.1 多核调度优化

【待完善】说明当前调度实现的基础（基于 ArceOS axtask）， 以及针对多核场景规划/已实施的优化（负载均衡、调度延迟等）。如工作尚未完成，说明设计方向和参考（KRR, OSDI'25）。

4.7.2 虚拟内存优化

【待完善】说明当前虚拟内存实现（VMA + 页表双层结构），以及参考 CortenMM（SOSP'25）取消独立 VMA 抽象的优化方向。 如工作尚未完成，说明设计思路。

4.8 本章小结

【待完善】总结本章的关键机制与性能优化工作，强调其在组件化兼容内核中的作用，并引出后续 AI 应用支持与实验评测。

第 5 章 AI 应用支持探索

【待完善】开头一段：说明 AIoT 场景下操作系统对 AI 应用支持的需求背景，以及本章工作的定位（探索性/规划性内容，目前处于进行中）。

5.1 目标平台与硬件环境

【待完善】简述目标开发板（配备 NPU 的边缘计算设备），说明需要完成的移植工作（StarryOS 跨架构支持、驱动适配）。

5.2 NPU 驱动设计

【待完善】简述 NPU 驱动开发的思路和挑战： 驱动模型、与内核内存管理的交互、DMA 支持等。

5.3 AI 应用运行支持

【待完善】简述目标应用（YOLO 目标检测、dora-rs 数据流框架）的运行需求，以及 StarryOS 需要提供的支持（推理框架兼容性、实时调度等）。

5.4 当前进展与后续计划

【待完善】如果工作尚未完成，可在此说明当前进展、阶段性结果及后续计划。

第 6 章 实验评测

【待完善】 开头一段：说明本章从功能完整性、应用兼容性和性能三个维度对 StarryOS 进行评测， 包括操作系统大赛标准测例、实际应用验证，以及关键性能基准测试。

6.1 实验环境

【待完善】简述测试所用的硬件平台（x86/riscv64/aarch64 等架构、QEMU 版本）和软件环境（Rust 工具链版本、Alpine Linux 镜像版本等）。

6.2 功能完整性评测

6.2.1 系统调用测例

【待完善】 介绍操作系统大赛的标准测例套件， 说明 StarryOS 的通过情况（可配表格，对比赛道测例通过率）。

6.2.2 应用兼容性测试

【待完善】 展示在 Alpine Linux 下运行实际应用的验证结果，包括：

- 开发工具链（GCC 编译、Cargo 构建、Git 操作）
- 图形应用（X11/dwm + ImageMagick 等）
- 其他典型应用（Python、SQLite、Redis、xv6 嵌套运行等）

可用截图或描述性文字。

6.3 性能评测

【待完善】 说明性能测试的方法和基准（lmbench、UnixBench 或自定义 microbenchmark）。

6.3.1 系统调用开销

【待完善】测试典型系统调用（getpid、read、write 等）的延迟， 与 Linux 基线对比。

6.3.2 I/O 性能

【待完善】测试文件读写吞吐量，体现异步机制和多级缓存优化的效果。

6.4 优化效果分析

【待完善】将第 4 章提出的关键机制与优化点和实验结果对应起来， 分析其对系统吞吐量、延迟和资源利用率的影响。

6.5 开发工作量统计

【待完善】统计整体工作量，参考 thuthesis-zyj 表 5.3 的形式： 主仓库新增代码行数、子仓库数量、commit 数量等。

6.6 本章小结

【待完善】总结实验结果，说明 StarryOS 在功能完整性、应用兼容性和性能方面的整体表现。

第 7 章 总结与展望

7.1 工作总结

【待完善】3-4 段，逐一总结各章的主要工作和成果：

1. 系统背景与总体架构：明确 StarryOS 的设计目标、系统定位与整体结构
2. 核心子系统实现：完成兼容性内核的基本盘，实现 200+ Linux 系统调用并支撑 Alpine 生态
3. 关键机制与性能优化：组件化设计探索（extern_trait、用户态切换）以及访存、 缓冲区、异步 I/O、多级缓存等优化
4. AI 应用支持探索：面向 AIoT 场景的 NPU 驱动与应用运行支持方向

7.2 未来工作展望

【待完善】2-3 段，说明后续计划：

- 多核调度优化（参考 KRR 等工作）
- 虚拟内存优化（参考 CortenMM，取消 VMA 抽象）
- NPU 驱动完善与 AI 应用落地
- 组件进一步开源复用（已有 weak-map、extern-trait、scope-local 等发布计划）

参考文献

[1] KLABNIK S, NICHOLS C. The Rust Programming Language[M]. No Starch Press, 2023.

[2] PANTER S, EISTY N. Rusty Linux: Advances in Rust for Linux Kernel Development[C]// Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM). 2024.

[3] LI Z, OTHERS. Rust for Linux: Understanding the Security Impact of Rust in the Linux Kernel[C]//2024 Annual Computer Security Applications Conference (ACSAC). 2024.

[4] CUTLER C, KAASHOEK M F, MORRIS R T. The Benefits and Costs of Writing a POSIX Kernel in a High-Level Language[C]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). 2018.

[5] LANKES S, PICKARTZ S, BREITBART J. HermitCore: A Unikernel for Extreme Scale Computing[C]//Proceedings of the 6th International Workshop on Runtime and Operating Systems for Supercomputers. 2016.

[6] LANKES S, OTHERS. RustyHermit: A Scalable, Rust-Based Virtual Execution Environment[C]//International Conference on High Performance Computing. Cham: Springer International Publishing, 2020.

[7] SHARMA A, OTHERS. Rust for Embedded Systems: Current State and Open Problems[C]// Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. 2024.

[8] LEVY A, OTHERS. Ownership is Theft: Experiences Building an Embedded OS in Rust[C]// Proceedings of the 8th Workshop on Programming Languages and Operating Systems. 2015.

[9] LEVY A, OTHERS. The Case for Writing a Kernel in Rust[C]//Proceedings of the 8th Asia-Pacific Workshop on Systems. 2017.

[10] LANKES S, BREITBART J, PICKARTZ S. Exploring Rust for Unikernel Development[C]// Proceedings of the 10th Workshop on Programming Languages and Operating Systems. 2019.

[11] KANTEE A. Flexible Operating System Internals: The Design and Implementation of the Anykernel and Rump Kernels[D]. 2012.

[12] PENG Y, OTHERS. Framekernel: A Safe and Efficient Kernel Architecture via Rust-based Intra-kernel Privilege Separation[C]//Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems. 2024.

[13] TIAN H, OTHERS. Asterinas: A Linux ABI-Compatible, Rust-Based Framekernel OS with a Small and Sound TCB[Z]. 2025.

[14] ZHANG J, OTHERS. CortenMM: Efficient Memory Management with Strong Correctness Guarantees[C]//Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP). 2025.

附录 A 外文资料的调研阅读报告（或书面翻译）

调研阅读报告题目（或书面翻译题目）

写出至少 5000 外文印刷字符的调研阅读报告或者书面翻译 1-2 篇（不少于 2 万外文印刷符）。

A.1 Lorem ipsum dolor.

A.1.1 Lorem ipsum dolor.

参考文献（或书面翻译对应的原文索引）

[1] 某某某. 信息技术与信息服务国际研讨会论文集: A 集［C］北京：中国社会科学出版社，1994.

附录 B 其他附录内容

【待完善】 填写其他附录内容（如有）

致　谢

【待完善】 填写致谢内容

声　明

本人郑重声明：所呈交的综合论文训练论文，是本人在导师指导下，独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签 名：_____ 日 期：_____

综合论文训练记录表

<table>
<tr><td>学生姓名</td><td>王铮</td><td>学号</td><td>2022010862</td><td>班级</td><td>计 26</td></tr>
<tr><td>论文题目</td><td colspan="5">AIoT 操作系统性能优化关键技术研究</td></tr>
<tr><td>主要内容以及进度安排</td><td colspan="5">主要内容：

进度安排：

指导教师签字：

考核组组长签字：

年　月　日</td></tr>
<tr><td>中期考核意见</td><td colspan="5">考核组组长签字：

年　月　日</td></tr>
</table>

指导教师评语	指导教师签字： 年　月　日
评阅教师评语	评阅教师签字： 年　月　日
答辩小组评语	答辩小组组长签字： 年　月　日

总成绩：_____

教学负责人签字：_____

年　月　日